Implementation and Benchmarking of Kyber and Dilithium

Sreejit Maity

October 2026

Introduction

We want to understand the difference between Post-Quantum Key Encapsulation (ML-KEM / Kyber) and Post-Quantum Digital Signatures (ML-DSA / Dilithium), and measure their timing behavior.

Key Concepts

- **Kyber (ML-KEM):** Used to safely exchange a secret key across the internet. It is based on the Learning With Errors (LWE) problem over polynomial rings.
- **Dilithium (ML-DSA):** Used to prove identity and sign documents so nobody can tamper with them. It uses the "Fiat-Shamir with Aborts" framework over module lattices.
- **Polynomial Multiplication:** The core math operation in both algorithms is multiplying polynomials modulo a prime q, typically speeded up using NTT (Number Theoretic Transform).

Benchmarking Comparison

Feature	**Kyber-512**	**Dilithium-2**
Type	Key Encapsulation (KEM)	Digital Signature
Public Key Size	800 bytes	1312 bytes
Ciphertext / Signature Size	768 bytes	2420 bytes
Primary Goal	Confidentiality	Authentication

Simplified C Implementation and Benchmark

The program below simulates polynomial ring multiplication (the building block of Kyber and Dilithium) and benchmarks key generation vs encapsulation.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define DEGREE 8
#define MODULUS 3329 // Kyber modulus (q)

// Simplified polynomial multiplication modulo (x^n + 1)
void poly_mul(int *a, int *b, int *result) {
    int temp[2 * DEGREE] = {0};
    for (int i = 0; i < DEGREE; i++) {
        for (int j = 0; j < DEGREE; j++) {
            temp[i + j] = (temp[i + j] + a[i] * b[j]) % MODULUS;
```

```
        }
    }
    // Reduce modulo x^DEGREE + 1
    for (int i = 0; i < DEGREE; i++) {
        result[i] = (temp[i] - temp[i + DEGREE] + MODULUS) % MODULUS;
    }
}

int main() {
    printf("--- Benchmarking Kyber vs Dilithium Core Math ---\n");

    int polyA[DEGREE] = {12, 45, 120, 3, 0, 99, 14, 5};
    int polyB[DEGREE] = {8, 19, 0, 44, 21, 10, 2, 70};
    int polyOut[DEGREE] = {0};

    // Benchmark 100,000 polynomial operations
    long iterations = 100000;
    clock_t start = clock();

    for (long i = 0; i < iterations; i++) {
        poly_mul(polyA, polyB, polyOut);
    }

    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Iterations : %ld\n", iterations);
    printf("Time taken : %f seconds\n", time_taken);
    printf("Sample coefficient [0]: %d\n\n", polyOut[0]);

    printf("Analysis:\n");
    printf("- Kyber relies on 2-3 matrix-vector poly products.\n");
    printf("- Dilithium requires rejection sampling, making sign slower than
        verify.\n");

    return 0;
}
```

Output

— Benchmarking Kyber vs Dilithium Core Math —
Iterations : 100000
Time taken : 0.073000 seconds
Sample coefficient [0]: 2212
Analysis:
- Kyber relies on $2-3$ matrix-vector poly products.
- Dilithium requires rejection sampling, making sign slower than verify.

Conclusion

Both Kyber and Dilithium are noticeably faster than RSA, but their keys and signatures are substantially larger. Lattice-based cryptography provides practical security against both conventional and quantum adversaries.